

Lab 1 — From a code change to a working endpoint

MLOps workshop · Instructor draft · 20 September 2026

Hands-on: about 25 minutes, plus training/deployment wait time.

Draft, not a live-tested procedure. These are proposed future participant actions. No repository, pipeline, Azure resource, or AML deployment was changed to prepare this handout. The instructor must pilot the flow before distribution.

Goal: edit Python in VS Code → commit and push → automatically train/evaluate/register → automatically deploy → send a prediction request.

Before starting

- Use your **generated application repository**, already cloned in VS Code, not the accelerator or reusable template repository. File links below are reference copies; edit your own clone.
- Assume your working branch is **main**, which selects the existing **prod** configuration. Here, prod must be **your assigned disposable workshop environment**, not a shared production service. Other branches select dev; do not switch branches casually.
- Your training and online pipelines already work and use the **same application repository and Azure DevOps project**. Your existing workspace, `cpu-cluster`, training environment, and endpoint are healthy. Keep existing identities, permissions, dependencies, infrastructure, and VM sizes.
- Use **one participant/team per workspace and endpoint**. Wait for the entire train-and-deploy cycle before the next push. The supplied model/data references use `latest`; overlapping runs can select another run's assets.
- The instructor has approved automatic runs, endpoint updates, and cost. Preserve branch policies and approval checks. If main requires a PR, use the approved PR workflow; the merge into main is the triggering push.

Record: application repository _____ · workspace _____ · endpoint _____

1. Capture the current prediction

In **Azure ML Studio** → **Endpoints** → **Online**, open your existing endpoint. Record the deployed `taxi-model` version. Open **Test**, paste the complete `data/test-request.json` from your clone, and select **Test**. Save the **two numeric predictions**, in input-row order. Keep this request unchanged for both labs; it contains 20 input features, not the target `cost`.

2. Make two small code edits in VS Code

Tune the model. In `data-science/src/train.py`, change only the default maximum depth from 10 to 12:

```
parser.add_argument('--regressor__max_depth', type=int, default=12,
                    help='Maximum number of levels in tree')
```

Make subsequent data comparisons repeatable. In `data-science/src/prep.py`, replace the line that creates `random_data` with:

```
random_data = np.random.default_rng(42).random(len(data))
```

Keep the existing split thresholds and all other code. This changes the old unseeded split, but the same row order now gives identical splits in Labs 1 and 2. Use Lab 1 as Lab 2's baseline. Training already fixes the forest's random seed. Save both files; do not push yet.

3. Enable training after every push to main

Open `mlops/devops-pipelines/deploy-model-training-pipeline.yml`. Replace its top-level `trigger: none` with:

```
trigger:
  batch: false
  branches:
    include:
      - main
```

Do not add a path filter: code **and** data changes must trigger training. In the existing `parameters` section, set these defaults so unattended CI reuses the working setup:

Existing parameter	default	Reason
<code>skipEnvironmentRegistration</code>	<code>true</code>	Reuse the healthy training environment.
<code>skipComputeCreation</code>	<code>true</code>	Do not run the compute-creation step.
<code>skipDataRegistration</code>	<code>false</code>	Upload/register the committed CSV, including Lab 2 edits.

Retain the existing job steps and required-success guards. A CI run uses these defaults; there is no manual parameter prompt.

4. Deploy only after the training pipeline succeeds

Open `mlops/devops-pipelines/deploy-online-endpoint-pipeline.yml`. **Keep its top-level `trigger: none`.** Replace its existing `resources` block with this block, substituting the source value:

```
resources:
  pipelines:
    - pipeline: training
      source: '\YOUR-APP-REPO\mlops\deploy-model-training-pipeline'
      trigger:
        branches:
          include:
            - refs/heads/main
  repositories:
    - repository: mlops-templates
      name: mlops-templates
      type: git
      ref: main
```

Important: `source` is the existing training pipeline's **exact Azure DevOps definition name, including its folder path and leading backslash**. The value shown assumes the initializer's application/mlops folder layout. Confirm it in **Pipelines** → **All**; do not copy `YOUR-APP-REPO` literally. The YAML's top-level `name` is a run-number setting, not proof of the definition's name.

Before pushing, have the instructor confirm both definitions use your application repository and have **Default branch for manual and scheduled builds = main / refs/heads/main**. YAML CI must not be disabled by a UI override. Authorize the referenced training pipeline if Azure DevOps requests permission; do not weaken project security.

Keep the online pipeline's existing **create/update** → **named-deployment smoke test** → **traffic allocation** steps. Do not give it a push trigger as well: that would start deployment before training finishes. Leave infrastructure and batch pipeline triggers unchanged.

5. Commit and push once

In VS Code **Source Control**, review and stage **only the four edited files**: training code, preparation code, training pipeline YAML, and online pipeline YAML. Commit with message **lab1: automate retraining and tune tree depth**, then **Push** to your authorized main branch.

Do not use `[skip ci]`. A local commit does not start Azure Pipelines: **the push does**. This configuration starts a CI run per qualifying push, not one run per commit bundled into that push. Do not click **Run pipeline** or push another change while this cycle is running.

6. Follow the change through MLOps

1. **Azure DevOps**: a training run appears automatically for your pushed commit. Record the commit and run URL. A resource-authorization prompt is not a successful run; resolve it with the instructor.
2. **AML Studio** → **Jobs**: open the matching `prod_taxi_fare_run_<build ID>` in experiment `prod_taxi_fare_train_main`. Confirm `prep_data` → `train_model` → `evaluate_model` → `register_model` complete. Under the training child's parameters, verify `max_depth = 12`. In the evaluation child, inspect `test_r2`, `test_rmse`, and `deploy_flag`.
3. **AML Studio** → **Models**: require a **new taxi-model version linked to this run** before claiming promotion. The gate compares the candidate with all previous versions on the current test set; changing code does not guarantee a better model.
4. **Azure DevOps**: after successful training, the online pipeline should start from the pipeline-completion trigger, using the **same application commit**. Its deployment and smoke test must succeed.

Do not mistake green for a new model. In the current repository, a rejected candidate skips registration without failing the training job. A completion trigger can therefore redeploy the **previous latest model**. If `deploy_flag = 0`, record **not promoted**; this is not completion of the new-model path. Do not force the flag or delete old models. Ask the instructor to review the candidate before another attempt.

7. Consume the updated endpoint

Return to **AML Studio** → **Endpoints** → **Online** → **your endpoint**. Verify deployment `taxi-online-dp` is **Succeeded**, its model version equals the accepted version from step 6, and traffic is **100%** after the smoke test.

Open **Test**, send the **same complete JSON** used in step 1, and save the two returned fares. The **endpoint URL stays the same**; the model behind it changes. Values may change, but version/lineage evidence matters more than expecting a particular fare.

Hand in	Your evidence
Source → training	Commit SHA, Azure DevOps run URL, matching AML job URL
Training → model	<code>max_depth = 12</code> , evaluation metrics, accepted model version
Model → endpoint	Automatic deployment run URL, deployed version, successful two-row response
Before → after	Original version/fares and new version/fares

Pass: the new model belongs to your commit's run, is the version actually deployed, and returns two numeric predictions. If registration is rejected or a task fails, stop and retain the evidence—do not report the old endpoint as the new result.

Safety: this updates the existing deployment in place; its current traffic is not isolated during an update. Use only the assigned sandbox endpoint. Leave the triggers in place for Lab 2. No resource deletion, resizing, package changes, or secret copying is part of this lab.

References: [Azure Repos CI triggers](#) · [Pipeline completion triggers](#). This deliberately small sandbox design uses `latest`; production should pass the exact approved model version between pipelines.